

AEGIS AGENT

Zero-Trust Tool Tokens and Scoped Authorization

MY IMPLEMENTATION-BASED ENGINEERING DOSSIER

I present the capability-scoped, time-limited tool-token component in my Aegis Agent project. I distinguish what the public source implements from what I have not verified. This is my engineering account of the implementation, not a penetration-test certification or a claim that every tool operation is protected.

01 | My architectural assessment

I implement an HMAC-SHA256-signed token format that carries tenant, task, tool, capability and expiration claims. In the application, I expose token issuance and validation endpoints. I regard these as concrete authorization primitives, not as proof of complete system-wide zero-trust enforcement: that stronger claim depends on issuer authentication, secret management and mandatory validation at every protected operation.

02 | What I can substantiate

• I can show in source: token creation, signature verification, expiry checks, tool-name checks, optional tenant/task matching, and required-capability matching.

• I can show in the API: tool issuance and validation endpoints in app/server.py.

• I can show in test definitions: unit tests for issuance/validation, missing capability, expiration and cross-tenant denial.

• I do not claim: comprehensive tool-execution interception, operational secret rotation, distributed revocation or passing CI results.

My primary code references

core/tool_tokens.py | app/server.py | tests/test_tool_tokens.py

03 | The trust boundary I implement

I structure the component around issuance and verification. The issuer signs canonical JSON claims; the verifier checks the signature and supplied contextual constraints before accepting a tool-related request. I use URL-safe Base64 as transport encoding, not encryption. I therefore do not treat token claims as confidential.

The claims I bind into each token

Field	Meaning
token_id	Per-token identifier derived from randomized seed
tenant_id / task_id	Logical scope of the token
tool_name	Named tool to which access is bound
capabilities	Explicit operations such as read
issued_at / expires_at	Time-bounded validity
signature	HMAC-SHA256 over canonical JSON

04 | How I issue a token

My issuer accepts tenant_id, task_id, tool_name and optional capabilities/TTL. It normalizes capabilities to lowercase, computes issue and expiry times, derives a token identifier using a randomized seed, then signs canonical JSON with HMAC-SHA256. I return the signed structure as URL-safe Base64. These steps describe the implementation, not an independently verified deployment.

05 | How I validate a token

I decode and parse the token, recalculate the HMAC over unsigned claims and compare signatures using hmac.compare_digest. I reject expired claims and mismatched tool names. I also enforce tenant and task matching when the verifier supplies those expected values, and require the requested capabilities to be present. I return a typed access object only after these checks.

06 | Where I integrate the API

In app/server.py, I register POST /api/v1/tools/issue and POST /tools/issue for token issuance. A tool-validation handler delegates to tool_token_manager.authorize_headers. I can substantiate these integration points in source; I cannot infer from them that all downstream tool executions are obligatorily intercepted.

07 | Security limitations I disclose

My token manager accepts an injected secret or AEGIS_TOOL_TOKEN_SECRET. If neither is supplied, the current source uses a fallback signing secret. I explicitly identify that as a security limitation; I do not publish its value here. I also do not claim trusted token issuance without demonstrating authentication and authorization at the issuance boundary.

08 | Denial behavior in my implementation

- Signature mismatch: invalid signature raises ToolTokenError.

• Expired token: expiry produces ToolTokenExpiredError.

• Tool mismatch: token cannot be used for a different named tool.

• Cross-tenant or cross-task: matching is enforced when the corresponding expected value is passed to validation.

• Missing capability: required permissions must be a subset of token capabilities.

• Missing token: authorize_request rejects an absent token.

09 | My illustrative engineering scenario

For illustration, I can issue a token scoped to tenant A, task 17, a repository-reader tool and the read capability. A verifier requiring write rejects that token; a verifier expecting tenant B rejects it as well. This is a conceptual example grounded in my validation logic, not a recorded production test.

My scope and failure-mode matrix

Condition	Observed behavior
Malformed or tampered token	Reject during decoding/signature validation
Expired claim	Reject after expiry comparison
Wrong tool / tenant / task	Reject when expected context is provided
Insufficient capabilities	Reject with missing-capability detail
Valid claims and signature	Return parsed access object

10 | What I know about the tests

I can point to four unit-test definitions in tests/test_tool_tokens.py: valid issuance/validation, missing-capability denial, expiry denial and cross-tenant denial. I reviewed their source, but I did not execute them for this publication. Accordingly, I make no claim about current pass rates, CI status or runtime reliability.

11 | Claims I deliberately do not make

• I do not claim that every tool invocation is mediated by these tokens.

• I do not claim a demonstrated revocation registry or complete key-rotation lifecycle.

• I do not claim confidentiality of token claims: Base64 is encoding, not encryption.

• I do not claim independently verified authorization of issuance across deployments.

• I do not claim that the test suite or CI currently passes.

12 | My engineering conclusion

In Aegis Agent, I implement an inspectable capability-token component and an HTTP integration surface. I use explicit scope claims, expiration and HMAC-based integrity checks. I present these as implemented primitives, while keeping their proven behavior separate from broader, unverified security guarantees.

My source and verification notes

My public source is github.com/aminazimi42-coder/aegis-agent. I based this dossier on core/tool_tokens.py, app/server.py and tests/test_tool_tokens.py. This was a read-only source review; I did not modify the repository or execute runtime tests, and I include no confidential credentials.